AI Compiler Performance Engineering

Auto-Loops Books — ch05

Contents

1 AMD XDNA/AIE Dataflow Architecture **1**

1.1 Why XDNA Is Not a Slower GPU 1
1.2 XDNA Tile Hardware: Fixed SRAM, No Runtime Scheduler 5
1.3 Static Dataflow: Four Rules Decode Must Obey 8
1.4 CUDA → XDNA Mapping for GPU Engineers 10
1.5 Online Softmax on Spatial Hardware 12
1.6 Architecture Tradeoffs and MoE Reality 13
1.7 Dataflow Inference Conclusions 15
1.8 YiRage XDNA Backend: Static Legality as Compile Gates 17
1.9 Code Review Gates (Merge Blockers) 20
1.10 XDNA Decode Benchmarking (Slim) 23
1.11 Key Takeaways 26
1.12 Conclusion 27

Bibliography **30**

Index **32**

Chapter 1

AMD XDNA/AIE Dataflow Architecture

1.1 Why XDNA Is Not a Slower GPU

A GPU team ported a Llama decoder to Ryzen AI and expected "CUDA, but slower." The graph either **failed to compile** or ran at DDR bandwidth with watt advantages erased (AMD, 2024b,a). Nsight-on-GPU habits misled them: there is no runtime warp scheduler, no configurable shared memory, and no silent spill to HBM when tile SRAM overflows (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

XDNA is a spatial dataflow array, not a SIMT machine with smaller scratchpad. Every buffer lifetime, DMA edge, and tile assignment must be fixed before the first token (Davies *et al.*, 2025; Chen, 2020). Illegal plans fail at compile time—or worse, export intermediates to DDR and lose the NPU's watt/token edge (Chen, 2026).

Bridge from Chapters 3–4. Chapter 3 listed the NPU row in the constraint matrix; Chapter 4 showed Hopper's *dynamic* tiers (shared memory, TMA, launches) (NVIDIA, 2022; SemiAnalysis, 2024). This chapter is the symmetric *static* story: tile SRAM, DMA chains, and shape buckets (AMD, 2024b,a). The decode metric lens is unchanged—TaxBreak's launches/token on GPU become DMA edges/token and DDR bytes/token here (Vellaisamy *et al.*, 2026; Chen, 2026).

Unified optimization logic. Whether the target is HBM or DDR, batch-1 decode still reads every weight and every KV vector each step (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Cut activation egress first; cut orchestration second; polish MAC utilization last (Chen, 2026; Vellaisamy *et al.*, 2026). On XDNA, "orchestration" is host graph submission plus DMA descriptor setup—not CUDA kernel launches (AMD, 2024b; Chen and YiRage Contributors, 2026).

What you will be able to do. After this chapter you can (1) explain why a Hopper-legal fusion may XDNA-reject, (2) write a tile SRAM line-item budget, (3) map TMA concepts to ping-pong DMA, (4) gate merges with five static-legality checks, and (5) benchmark DDR bytes/token instead of FLOPs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Reader prerequisites. Carry Chapter 4's priority pyramid mindset but invert flexibility for determinism (Chen, 2026). When we reference online softmax, Chapter 10 owns numerics; here we cover *where* carriers must live in tile SRAM (Dao *et al.*, 2022).

Industrial narrative. TaxBreak dense Llama-3.2-1B averages hundreds of GPU kernel launches per output token on H100 (Vellaisamy *et al.*, 2026). XDNA cannot budget "847 launches"—each logical launch must become a static DMA edge or disappear in fusion (AMD, 2024b; Chen and YiRage Contributors, 2026). Memory-Floor shows CUDA Graphs removing host overhead on GPU; on XDNA the comparable win is eliminating redundant DDR round trips, not replaying kernels (Chen, 2026; NVIDIA, 2024).

Silicon context. Ryzen AI combines Zen cores with XDNA tiles aimed at laptop inference (AMD, 2024b,a). Decode workloads stress DDR interfaces shared with CPU graphics and I/O—contention shows up as tail latency, not average MAC utilization (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Planning must include system-level bandwidth, not isolated NPU peaks (Chen and YiRage Contributors, 2026; Chen, 2026).

Anti-pattern preview. The five review gates in Section 1.9 map to static-legality failures—use them in design review before merge (Chen and YiRage Contributors, 2026; AMD, 2024a). Culture: post Table 1.2 in your compiler team's XDNA doc template (Chen, 2020).

Cross-chapter dependency map. Chapter 3 listed abstract NPU constraints; Chapter 4 named Hopper mechanisms; Chapters 6–11 specialize residency, sync, softmax, and MegaKernel codegen (Chen, 2020; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). Skipping here produces schedules that ignore tile SRAM bounds (AMD, 2024b; Chen, 2026).

How to read this chapter. Skim Table 1.3 first if you are a GPU engineer (Davies *et al.*, 2025). Sections 1.2–1.3 are the hardware contract; Section 1.9 is the merge workflow; Section 1.10 is verification only (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Organization map. Tiles (§1.2), static rules (§1.3), CUDA mapping (§1.4), softmax (§1.5), tradeoffs (§1.6), dataflow conclusions (§1.7), YiRage (§1.8), gates (§1.9), verify (§1.10) (Chen and YiRage Contributors, 2026; AMD, 2024b).

What this chapter is not. Not a Ryzen AI marketing brief; not a substitute for AMD vendor programming guides (AMD, 2024b,a). We document optimization *bounds* for batch-1 decode residency on spatial dataflow silicon (Chen and YiRage Contributors, 2026; Chen, 2020, 2026).

Failure story catalog (short). Tile SRAM overflow at max ctx; illegal DMA on paged KV; bucket mismatch after driver upgrade; softmax carrier width drift; host alloc sneaking into hot path (Chen and YiRage Contributors, 2026; AMD, 2024b; Dao *et al.*, 2023; Davies *et al.*, 2025; Kwon *et al.*, 2023). Map symptoms to static rules in Table 1.2 before tuning MAC tile sizes (Chen, 2026; Vellaisamy *et al.*, 2026).

Teaching versus production stacks. Courses emphasize rectangular GEMM throughput; production decode emphasizes DDR bytes/token and watt/token counters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026). This chapter bridges that gap for XDNA specifically (AMD, 2024a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

DeepSeek deployment reminder. Decode pools dominate GPU counts in large deployments; edge assistants invert the story—watt caps and locality dominate (Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen,

2026). Pick silicon against the binding metric, not die marketing (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Priority contrast with Chapter 4. Chapter 4's pyramid starts with fusion and graphs, then TMA, then TC polish (Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). XDNA's pyramid is stricter: static legality and DDR bytes/token first—MAC polish is still last, but compile failure blocks all runtime tuning (AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Never invert the order by opening MAC configurators while buffer tables overflow (AMD, 2024a; Chen, 2026).

Who should read this chapter. GPU kernel engineers porting to Ryzen AI, compiler authors lowering MLIR to AIE, and serving teams splitting cloud GPU pools from edge NPU nodes (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2022). CUDA-only deployers should still read the mapping and tradeoff sections—they explain why GPU recipes fail on NPU fleets (AMD, 2024b; Chen, 2020).

Reader exercises. (1) Sketch tile SRAM allocation for one fused layer at d=2048. (2) Explain why Hopper TMA overlap does not reduce DDR bytes. (3) Fill Table 1.7 for your model (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; AMD, 2024a).

Onboarding sentence for teams. Post in your compiler design doc: *"We do not merge XDNA decode changes without static buffer tables, shape buckets, and DDR goodput counters."* (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b) CI enforces the sentence; posters alone do not (Davies *et al.*, 2025; Chen, 2020).

Open research threads. Dynamic shapes without recompile, multi-chip XDNA graphs, and MoE expert-local fusion remain active (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024a). This chapter documents single-NPU batch-1 baselines engineers can extend (Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b).

Industrial narrative (closing hook). Goodput engineering on XDNA is measurable: DDR bytes/token, DMA edges/token, ms/token, and watt/token together

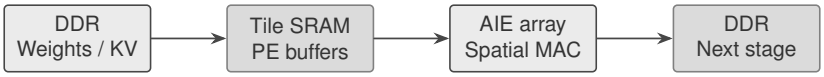


Figure 1.1: XDNA decode data movement: DDR ingress dominates when tile residency fails.

Table 1.1: XDNA tile resources vs. Hopper SM (decode comparison).

Resource	**Hopper SM**	**XDNA AIE tile**
On-chip scratch	Configurable shared mem	Fixed tile SRAM
Parallel grain	Warp (32 threads)	Spatial PE lane
Scheduling	Runtime + streams	Compile-time DMA graph
Dynamic shapes	Supported (with cost)	Bucketed / recompile
Failure mode	Launch storms, spill	Compile fail, DDR egress

(Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024b). Skillful static residency beats SKU shopping when DDR roofline binds (Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; AMD, 2024a).

1.2 XDNA Tile Hardware: Fixed SRAM, No Runtime Scheduler

AMD **XDNA** groups AI Engine (**AIE**) **tiles** into a spatial array (AMD, 2024b,a). Each tile owns local SRAM, connects through stream switches, and executes MAC datapaths under compiler-scheduled DMA—not under a hardware warp scheduler (Davies *et al.*, 2025).

Engineering meaning. Tile SRAM is smaller than Hopper shared memory and *fixed* at compile time—there is no "configure 228 KB and hope" knob (NVIDIA, 2022; AMD, 2024a). Overflow does not spill gracefully; it illegalizes the graph or forces DDR round trips (Chen and YiRage Contributors, 2026; Chen, 2026).

Common pitfall. Sizing tiles from GPU comfort zones ("HBM is huge") instead of per-SKU SRAM datasheets (AMD, 2024b). **Debug:** static buffer table high-water versus tile cap; DDR bytes/token rising while MAC active % stays flat (Davies *et al.*, 2025).

Core bottleneck. When weights, KV cache, and live intermediates cannot simultaneously fit tile SRAM, decode becomes DDR-bandwidth bound—the NPU roofline collapses to external memory (Chen, 2026; Dao *et al.*, 2022). Chapter 2's export-edge mindset applies: every DDR round trip is a bytes/token bill (Chen, 2020).

Ryzen AI context. Ryzen AI pairs Zen cores with XDNA for laptop and edge inference (AMD, 2024b). DDR is shared with CPU, graphics, and I/O—contention inflates p99 decode even when isolated NPU peaks look healthy (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Decode vs prefill. Prefill parallelizes over prompt tokens; decode appends one token with full KV history (Dao *et al.*, 2022, 2023). Graphs compiled for rectangular prefill matmuls may waste tile arrays on thin $M{=}1$ decode—qualify decode-shaped buckets separately (AMD, 2024a; Chen and YiRage Contributors, 2026).

Example 1.2.1. A decode matmul streams weights from DDR into tile SRAM while activations ping-pong on a DMA chain. Two weight tiles with overlapping lifetimes exceed SRAM—compile fails with no HBM spill fallback (Chen and YiRage Contributors, 2026; AMD, 2024b).

Worked contrast. Hopper may recover from a suboptimal schedule with CUDA Graphs and extra launches (NVIDIA, 2024; Vellaisamy *et al.*, 2026). XDNA rarely gets that slack: the static graph ships or it does not (AMD, 2024a; Chen and YiRage Contributors, 2026).

Tile array topology. Multiple AIE tiles connect in a grid; decode megakernels assign pipeline stages to tile columns or rows (AMD, 2024b,a). Compiler passes must respect inter-tile stream-switch bandwidth—not every fusion that fits one tile fits the array (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Oversubscription may not fail compile; it stalls runtime and inflates p99 (Chen, 2026).

Weight streaming. Weights are read every decode token (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). On GPU, weight bandwidth competes with KV in HBM; on XDNA, weights DMA from DDR unless hot slices pin in tile SRAM across

tokens (Chen, 2026; AMD, 2024b). Plans that reload full layers each token saturate DDR regardless of MAC peak (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

KV on DDR. KV typically lives in DDR on edge NPUs; attention reads history each token (Kwon *et al.*, 2023; Dao *et al.*, 2023). Paged KV still requires static page tables in many XDNA stacks (Chen and YiRage Contributors, 2026; Chen, 2026). Qualify with bytes/token, not parameter count alone (Vellaisamy *et al.*, 2026).

Rotary and norms. Layer norm and rotary embedding look tiny in FLOPs; unfused they become separate DMA rounds (Vellaisamy *et al.*, 2026; AMD, 2024a). Fuse into attention or MLP stages where carriers are already resident (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Quantization note. INT8 and INT4 weights reduce DDR pressure—critical on XDNA (AMD, 2024b; Chen, 2026). Quantization scales must embed in static graphs; runtime per-op scale lookup reintroduces dynamicism (AMD, 2024a; Chen and YiRage Contributors, 2026).

Occupancy versus fusion trade. Fewer resident tiles may raise MAC active % yet lose on ms/token when fusion exports to DDR (Chen, 2026; Davies *et al.*, 2025). Optimize MAC charts only after DDR bytes/token improves (Chen and YiRage Contributors, 2026).

Tier assignment exercise. For one transformer layer, list every $\Theta(d)$ tensor between attention and MLP: post-norm hidden, Q/K/V fragments, attention output, MLP intermediates, residuals (Dao *et al.*, 2022; Chen, 2026). Mark each as tile SRAM, DDR staging, or illegal export for your XDNA plan (Chen and YiRage Contributors, 2026; Chen, 2020). Any DDR mark between fused stages is a bytes/token bug unless proven dead (Vellaisamy *et al.*, 2026; Chen, 2026).

✓ **Practice checkpoint.** Attach a tile SRAM line-item sheet listing every buffer alive during one decode step before requesting codegen (Chen and YiRage Contributors, 2026; Chen, 2026).

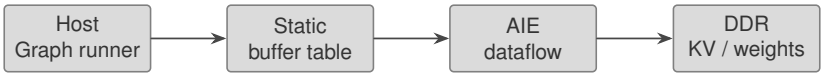


Figure 1.2: Static dataflow: every buffer lifetime is fixed before decode begins.

Table 1.2: Static dataflow iron rules for XDNA decode.

Rule	**Engineering requirement**
No dynamic alloc	Per-token hot path uses compile-time DDR slots; KV growth aliases pre-planned regions (Kwon *et al.*, 2023).
No naked dynamic CF	Runtime branches become padded static bucket variants (Davies *et al.*, 2025).
Deterministic DMA	Producer–consumer edges map to DMA events with known latency—not "sync later" (AMD, 2024a).
Fusion all-or-nothing	Partial fusion exporting intermediates to DDR erases watt advantage (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

1.3 Static Dataflow: Four Rules Decode Must Obey

Static XDNA schedules resolve every tensor lifetime, DMA descriptor, and tile assignment at compile time (AMD, 2024b,a). **Dataflow** propagates tokens through spatial pipelines—not through a sequence of host-launched kernels (Chen, 2020; Chen and YiRage Contributors, 2026).

Host responsibilities. The host configures graph runners and submits execute calls—but must not allocate full tensors per token (AMD, 2024b). When KV exceeds compiled max context, the product recompiles or rejects—not fall back to unplanned buffers (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Synchronization model. GPUs misuse `cudaDeviceSynchronize`; XDNA relies on DMA completion tokens wired in the graph (AMD, 2024a; Davies *et al.*, 2025). Host waits between stages add jitter to p99 decode (Chen, 2026).

Deployment pitfall. Benchmarking prefill-length graphs then deploying batch-1 decode with growing KV violates static buffer tables unless context was bucketed upfront (Dao *et al.*, 2022, 2023). Treat ctx buckets as compile parameters, not runtime surprises (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Recompile economics. Static specialization amortizes compile minutes over thousands of tokens (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Edge products with cold starts must budget compile latency in UX—unlike GPU JIT hidden in first batch (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Chapter 2 iron rule link. Iron Rule 2 (keep activations on-chip) becomes a *hard* compile gate on XDNA—violations export to DDR, not to a forgiving spill path (Chen, 2020, 2026).

Compiler IR implications. MLIR and YiRage attach static lifetime intervals to memref views before lowering to AIE (Chen and YiRage Contributors, 2026; Chen, 2020). Generic bufferization that inserts heap fallback breaks static rules immediately (Chen, 2026; Kwon *et al.*, 2023).

Testing static graphs. Validate at min and max context buckets, not only nominal length (Dao *et al.*, 2023; AMD, 2024a). Off-by-one carrier sizes often appear only at max context (Chen, 2026). Run DDR byte counters in simulation before silicon (AMD, 2024b).

Continuous batching. vLLM-style batching on GPU does not port literally—XDNA batching requires padded static batch classes (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026). Each batch class is a separate compiled graph with its own buffer table (Davies *et al.*, 2025).

PyTorch eager anti-pattern. Chapter 1 diagnosed operator-first GPU stacks; on XDNA eager dispatch is often unsupported outright (Vellaisamy *et al.*, 2026; AMD, 2024b). The compiler must emit one static graph per shape bucket and

Table 1.3: CUDA/Hopper → XDNA/AIE concept mapping (decode).

CUDA / Hopper	**XDNA / AIE**
TMA async copy	DMA load into tile SRAM
Shared memory tile	Tile buffer + ping-pong DMA chain
CUDA Graph capture	Static dataflow graph
Stream kernel launch	Host graph runner single submit
Warp online softmax	Spatial softmax carrier pipeline

recompile when buckets change (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Static dataflow worked example. Imagine a two-stage decode cell: attention then MLP (Dao *et al.*, 2022). Stage one DMA-loads K/V while MAC arrays update softmax carriers entirely in tile SRAM. Stage two consumes attention output from stream registers without a DDR round trip (AMD, 2024a; SemiAnalysis, 2024). If MLP weights DMA mid-stage without double buffering, MAC units stall—visible as higher ms/token despite healthy FLOPs counters (Davies *et al.*, 2025; Chen, 2026).

✓ **Practice checkpoint.** Every merge attaches a buffer lifetime diagram: which tensors coexist in tile SRAM at each decode step (AMD, 2024a; Chen and YiRage Contributors, 2026).

1.4 CUDA → XDNA Mapping for GPU Engineers

GPU engineers think in TMA descriptors and shared-memory tiles; XDNA engineers think in **buffer descriptor**s and tile SRAM assignments (NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b). The mapping is *conceptual*—CUDA scheduling experience does not transplant (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

TMA lesson transfer. TMA hides HBM latency while MMA computes (SemiAnalysis, 2024; NVIDIA, 2022). XDNA needs explicit ping-pong buffer descriptor pairs in the static graph—compiler assigns both buffers and DMA edges (AMD,

2024a; Chen and YiRage Contributors, 2026). Without overlap, MAC utilization collapses and watt/token rises (Davies *et al.*, 2025).

Graph capture analogy. CUDA Graphs freeze launch order; XDNA freezes DMA order (NVIDIA, 2024; AMD, 2024b). Mutating either without re-record/re-compile breaks decode silently (Vellaisamy *et al.*, 2026; Chen, 2026).

Applicable when. Regular tiles, amortized descriptor setup, steady MegaKernel pipelines inside one shape bucket (AMD, 2024b; SemiAnalysis, 2024).

Avoid when. Expecting dynamic batching, speculative branches, or per-token `malloc` on the hot path (Davies *et al.*, 2025; Liu *et al.*, 2024).

Common pitfall. Mandating DMA on micro-copies where descriptor tax exceeds copy benefit—mirror Chapter 4's thin-M TMA anti-pattern on DDR (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Debug. MAC active % versus DDR throughput; DMA edges/token versus graph submission time (AMD, 2024a; Davies *et al.*, 2025).

Example 1.4.1. GPU: TMA loads Q while softmax partials stay in shared memory. XDNA: DMA chain loads Q into tile A while tile B holds running max/sum carriers—roles swap at compile-time ping-pong, not runtime alloc (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

Double buffering depth. Hopper TMA double buffering hides HBM latency; XDNA ping-pong DMA chains require explicit buffer descriptor pairs in the static graph (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026). Shallow pipelines under-utilize MAC arrays; deep pipelines overflow tile SRAM (Chen, 2026; Davies *et al.*, 2025).

Buffer descriptors in practice. A buffer descriptor names source address, destination tile buffer, length, and stride—analogous to TMA copy descriptors on Hopper (NVIDIA, 2022; AMD, 2024b). Ping-pong uses two descriptors alternating roles each token step (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023). Decode graphs freeze descriptor contents per shape bucket; changing context without recompile implies descriptors were padded for max context upfront (Davies *et al.*, 2025; AMD, 2024a).

Host orchestration. CUDA uses streams; XDNA uses graph runners that submit entire dataflow frames (NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026). Per-token host work must be O(1) and allocation-free—TaxBreak's kernel launch overhead has a direct analogue in graph submission overhead (Vellaisamy *et al.*, 2026; Chen, 2026).

✓ **Practice checkpoint.** Diff static buffer tables against tile SRAM caps; list every DDR transaction per decode step (Chen, 2026; AMD, 2024a).

1.5 Online Softmax on Spatial Hardware

Attention decode needs numerically stable **softmax** over growing KV context (Dao *et al.*, 2022, 2023). FlashAttention-style algorithms keep running max and sum in on-chip **carriers** rather than materializing full attention maps to DDR (Chen, 2026).

Hardware requirement. On GPU, carriers live in shared memory with warp-synchronous updates; on XDNA they need fixed tile buffer *slots* in the static dataflow (AMD, 2024a; Chen and YiRage Contributors, 2026). A standalone softmax op lowered independently forces a full map to DDR—doubling bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026).

Applicable when. $QK^\top$, carrier update, and PV accumulate fuse in one spatial pipeline with carriers never leaving tile SRAM (Dao *et al.*, 2022; SemiAnalysis, 2024).

Avoid when. Importing unfused GPU softmax kernels "because numerics are already validated" (AMD, 2024b; Chen and YiRage Contributors, 2026). The rejection is correct—residency failed, not the NPU (Davies *et al.*, 2025).

Common pitfall. Carrier buffers sized for prefill-length vectors when decode KV tiles are smaller—overflow splits the graph into DDR-bound pieces (Dao *et al.*, 2023; Chen, 2026).

Debug. Compare logits against GPU reference at short ctx before trusting long ctx; inspect carrier placement in cycle-accurate simulation (AMD, 2024a; Chen and YiRage Contributors, 2026).

Carrier precision. Fixed-point carriers save SRAM but risk saturation at long context (Dao *et al.*, 2022). Compiler search must joint-optimize placement and numeric format per tile budget (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Worked contrast. Hopper Flash-Decoding adds sequence parallelism with more blocks (Dao *et al.*, 2023). XDNA lacks a "launch more blocks" escape—sequence parallelism must be spatial lanes or compile-time rejection (AMD, 2024b; Chen and YiRage Contributors, 2026).

KV loop structure. For each KV block: DMA K/V tiles, MAC $QK^\top$, online softmax carrier update, MAC PV accumulate (Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023). The loop body must stay inside the same static graph as QKV and MLP—exporting partial attention between blocks reopens DDR bytes/token (Chen, 2026; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Numerical stability. Online softmax relies on stable accumulation order; spatial reductions must preserve mathematical equivalence to GPU reference implementations (Dao *et al.*, 2022, 2023). Validation should compare logits against GPU reference on short contexts before trusting long-context deployments (Chen, 2026; AMD, 2024a).

Carrier placement. Assign carrier buffers adjacent to attention tiles in the dataflow to minimize stream latency (AMD, 2024b; Davies *et al.*, 2025; Chen, 2020). Poor placement increases stage cycles without changing FLOPs—visible only in cycle-accurate simulation (Chen and YiRage Contributors, 2026; AMD, 2024a).

✓ **Practice checkpoint.** Softmax region must show fused carrier updates in the static graph—no DDR materialization edge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

1.6 Architecture Tradeoffs and MoE Reality

CUDA decode trades launches against bytes: graphs and fusion cut orchestration; TMA and shared memory cut HBM traffic (NVIDIA, 2024; Vellaisamy *et al.*, 2026;

Table 1.4: Decode tradeoffs: CUDA (Hopper) vs XDNA (AIE), batch-1.

Dimension	**CUDA (Hopper)**	**XDNA (AIE)**
Orchestration tax	Launches/token	Graph submit + DMA setup
Flexibility	High	Low (static buckets)
Peak memory path	HBM + TMA	DDR + tile SRAM
Primary win metric	ms/token (cloud)	watt/token (edge)
MoE decode	Launch storms	Static routing required
Best fit	Elastic cloud pools	Watt-capped edge

SemiAnalysis, 2024). **XDNA** trades compile-time staticism against DDR egress: watt/token wins when the decoder step is one spatial pipeline (AMD, 2024b,a; Chen, 2026).

When XDNA wins. Fixed-model edge assistants with bounded context, watt caps, and fully static megakernels (AMD, 2024b; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

When CUDA wins. Dynamic batching, speculative decoding, long-context cloud serving, frequent graph changes (Kwon *et al.*, 2023; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

MoE starkest split. TaxBreak reports thousands of GPU kernel launches per MoE decode token (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). XDNA cannot absorb that count—experts must fuse, batch, or compile as separate static graphs with explicit routing DMA (Chen and YiRage Contributors, 2026; AMD, 2024a). Choosing XDNA for MoE without a static routing plan repeats GPU orchestration failure in a stricter legality environment (Davies *et al.*, 2025; Chen, 2026).

Hybrid fleets. Production may run GPU decode pools plus XDNA edge nodes—schedules must diverge by design, not by accident (Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026).

Speculative decoding caveat. Speculative paths introduce dynamic branches hostile to static graphs (Davies *et al.*, 2025; Liu *et al.*, 2024). Compile separate graphs per policy or keep speculation on GPU (Chen and YiRage Contributors, 2026; AMD, 2024b).

Software stack depth. Ryzen AI ships runtime, compiler, and drivers—teams must coordinate versions like CUDA toolkit pins (AMD, 2024a,b; Chen, 2020). Upgrading one layer without requalifying decode graphs repeats familiar MLIR upgrade failures (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026).

Power envelopes. Watt/token often drives XDNA product wins even when raw ms/token loses to H100 (AMD, 2024b,a; Chen, 2026; Davies *et al.*, 2025). Architecture reviews should publish both metrics when fleets mix silicon (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Observability. Instrument DMA bytes, MAC active ratio, and graph submission time separately (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026). Without decomposition, teams misallocate effort to weight quantization when scheduling is the bottleneck (Chen and YiRage Contributors, 2026; AMD, 2024a).

Multi-layer decode step. A full decoder step chains layer graphs or one fused multi-layer graph (Chen and YiRage Contributors, 2026; AMD, 2024a). Chaining increases host submissions; fusing increases SRAM pressure (Davies *et al.*, 2025; Chen, 2026). Autotune Pareto fronts across those axes rather than optimizing one layer in isolation (Chen and YiRage Contributors, 2026; Chen, 2020).

✓ **Practice checkpoint.** Architecture review picks primary metric—watt/token or ms/token—before silicon selection (AMD, 2024b; Davies *et al.*, 2025).

1.7 Dataflow Inference Conclusions

Dataflow is the operational model for NPU inference when decode must meet watt and latency envelopes (AMD, 2024b; Chen, 2020; Chen and YiRage Contributors, 2026).

1. Treat decode as a static MegaKernel per shape bucket; recompile when buckets change (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).
2. Measure DDR bytes/token and DMA edges/token—not FLOPs (Chen, 2026; Vellaisamy *et al.*, 2026).

3. Reject graphs that materialize KV or softmax intermediates to DDR (Dao *et al.*, 2022, 2023).

4. Pair GPU and XDNA baselines from the same high-level IR (Chen, 2020; Chen and YiRage Contributors, 2026).

Operator runbooks. List compiled shape buckets, max context, recompile triggers, and expected DDR bytes/token (Davies *et al.*, 2025; AMD, 2024b). Operators otherwise misdiagnose KV growth as model quality issues (Kwon *et al.*, 2023; Chen, 2026).

Failure modes (symptom → counter). Compile rejection → buffer table overflow row (Chen and YiRage Contributors, 2026; AMD, 2024a). DDR saturation → bytes/token versus roofline (Chen, 2026). Host jitter → p99 graph submission time (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Identical weights do not imply identical schedules across GPU and NPU fleets (Liu *et al.*, 2024; Davies *et al.*, 2025). Compiler stacks must fork early; "port the CUDA kernel" without static replanning fails (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Epilogue for practitioners. XDNA rewards teams that think like compiler engineers first and MAC micro-optimizers second (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020). Static dataflow is the price of watt-efficient decode; pay it deliberately (Chen, 2026; Davies *et al.*, 2025). When graphs fail, celebrate early rejection—it is cheaper than DDR-bound production (Chen and YiRage Contributors, 2026; AMD, 2024a).

Roadmap to later chapters. Chapter 8 specializes async copy theory on GPU; Chapter 9 sync hierarchy; Chapter 10 softmax numerics; Chapter 11 MegaKernel assembly (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). XDNA bounds here are prerequisites—later chapters reference GPU labels while NPU teams keep static buffer tables parallel (AMD, 2024b; SemiAnalysis, 2024; NVIDIA, 2022).

Energy and TCO (short). Fusion DDR bytes/token savings reduce external memory energy even when ms/token gains look modest (Chen, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024b). Edge TCO cares about joules per

useful token—not peak TOPS banners (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Postmortem sequence. When XDNA decode regresses after a merge: diff DDR bytes/token, replay buffer table high-water, compare DMA edge count, bisect fusion versus bucket change, *then* consider driver issues (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; AMD, 2024a; Chen and YiRage Contributors, 2026). Skipping bytes/token replay wastes weeks on MAC-only noise (Chen, 2020).

1.8 YiRage XDNA Backend: Static Legality as Compile Gates

Advance notice. This section assumes compiler and MLIR literacy; backend engineers read end-to-end, kernel tuners skim invariants (Chen, 2020; Chen and YiRage Contributors, 2026).

YiRage targets **XDNA** with the same residency vocabulary as its CUDA backend: buffer lifetimes, fusion boundaries, and decode metrics (Chen and YiRage Contributors, 2026; AMD, 2024b,a).

Five compiler invariants (XDNA decode mode).

1. **No hot-path heap alloc**—*YiRage:* `BufferizationPass` binds KV and temporaries to static DDR slots (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

2. **Tile SRAM high-water**—*YiRage:* `TileAssignmentPass` fails compile when simultaneous buffers exceed cap (AMD, 2024a; Chen, 2026).

3. **DMA legality per edge**—*YiRage:* `DmaSchedulingPass` emits deterministic producer–consumer events (AMD, 2024b; SemiAnalysis, 2024).

4. **Softmax carriers fused**—*YiRage:* `CarrierFusionPass` rejects full-map materialization (Dao *et al.*, 2022, 2023).

5. **Triplet bufferization**—*YiRage:* same IR must export legality summaries for CUDA, CPU, and XDNA (Chen, 2020; Davies *et al.*, 2025).

Table 1.5: YiRage XDNA lowering: IR in, check out.

Pass	**Input → action**	**Output check**
MLIR import	Framework export → attach tile/DMA metadata	Chip model from Ch. 3
Bufferization	Tensor IR → static DDR + tile slots	No hot-path alloc
Fusion grouping	Graph → export-edge merges	No DDR activation stores
Tile assignment	Fused region → spatial map	SRAM high-water ≤ cap
DMA schedule	Copy edges → ping-pong chains	Deterministic latency
Graph emit	Scheduled IR → AIE binary	Triplet legality report

Multi-backend CI. YiRage CI compiles the same decoder export for CUDA and XDNA on every merge, publishing legality diffs (Chen and YiRage Contributors, 2026; Chen, 2020). When XDNA compile fails, errors cite buffer table rows and tile IDs—not opaque backend codes (AMD, 2024a; Davies *et al.*, 2025).

Toolchain and chip-model pins. XDNA legality depends on the AIE toolchain and the chip model your YiRage backend loads, not just the graph IR (AMD, 2024a; Chen and YiRage Contributors, 2026). A graph that fits the current tile map can change legality after a driver or firmware bump; pin the toolchain and chip-model commit in decode CI and re-run the five gates on every upgrade (AMD, 2024b; Davies *et al.*, 2025). Treat a silent legality flip as a compile error, not a benchmark curiosity—the same IR export must remain shippable across the fleet before any performance claim is compared against the pinned baseline (Chen and YiRage Contributors, 2026; AMD, 2024a).

IREE-style portability. Modular retargeting treats backends as plugins over shared IR (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). XDNA tile capacities live in the plugin chip model, not scattered in hand-written kernels (Davies *et al.*, 2025; AMD, 2024b,a).

Lowering walkthrough detail. Each row in Table 1.5 must emit artifacts CI can diff—legality reports are not optional debug prints (Chen and YiRage Con-

tributors, 2026; Chen, 2020; AMD, 2024a). Bufferization without tile assignment is incomplete; DMA schedule without carrier fusion is unsafe for attention decode (Dao *et al.*, 2022; Chen, 2026; Davies *et al.*, 2025).

Testing gates. Numerical diffs at long ctx; DDR byte delta at batch-1; triplet compile; bucket hit rate across three ctx lengths (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026).

Debug workflow. On failure, dump static buffer table and DMA edge list before reading assembly (Chen and YiRage Contributors, 2026; AMD, 2024b). Regression tests compile canonical decoder layers per supported Ryzen AI SKU and fail on new allocs in the hot path (AMD, 2024a; Chen, 2020).

Example 1.8.1. YiRage merge gate: Llama-3.2-1B decoder layer at BS=1, ctx=2048 must compile as one static XDNA graph or document failing ops; attach GPU launches/token for the same IR export (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026).

Documentation deliverables per promotion. Residency diagram, DMA edge list, shape bucket map, softmax carrier changelog, DDR bytes/token cell, watt/token cell (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Partners cannot replay regressions when benchmarks live only in private spreadsheets (Chen, 2020; AMD, 2024b).

CUDA Graph parity note. Chapter 4 fused Hopper kernels pair with CUDA Graph capture; XDNA pairs fusion with static graph emit (NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026). Triplet CI should verify both capture replay *and* static graph legality on the same IR export (Chen, 2020; Vellaisamy *et al.*, 2026).

Silent DDR export signature. Flat MAC utilization with rising DDR throughput often means partial fusion or carrier spill (Chen, 2026; AMD, 2024a; Davies *et al.*, 2025). Map telemetry to static rules in Table 1.2 before opening weight-format experiments (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Triplet export compiles or documents deferral with buffer table attached (Chen and YiRage Contributors, 2026; AMD, 2024b).

Table 1.6: Tile SRAM budget template (Llama-class layer, illustrative).

Buffer	**Tier**	**Bytes**	**Notes**
Q slice	Tile SRAM	4 KB	One decode step
K/V tile	Tile SRAM	16 KB	Ping-pong DMA
Softmax carriers	Tile SRAM	2 KB	(m, ℓ) per head tile
MLP fragment	Tile SRAM	12 KB	Fused with attention out
Subtotal		**34 KB**	Must fit per-tile cap

1.9 Code Review Gates (Merge Blockers)

1. **Tile SRAM budget.** *Symptom:* compile pass on GPU, fail on XDNA. *Debug:* simultaneous buffer sum versus per-tile cap (AMD, 2024a; Chen and YiRage Contributors, 2026).

2. **Static graph compliance.** *Symptom:* runtime shape mismatch or silent recompile storms. *Debug:* hot path has no dynamic dims without bucket recompile (Davies *et al.*, 2025; Kwon *et al.*, 2023).

3. **DDR ledger.** *Symptom:* NPU "feels slow" despite healthy MAC %. *Debug:* DDR bytes/token vs GPU baseline on same model (Chen, 2026; Vellaisamy *et al.*, 2026).

4. **Softmax fusion.** *Symptom:* attention bytes/token doubles at long ctx. *Debug:* no full-map DDR edge in static graph (Dao *et al.*, 2022, 2023).

5. **Triplet IR consistency.** *Symptom:* CUDA-only schedule ships to heterogeneous fleet. *Debug:* same export compiles on CUDA and XDNA backends (Chen and YiRage Contributors, 2026; Chen, 2020).

✓ **Practice checkpoint.** All five gates green before merge; attach Table 1.6 or SKU-specific equivalent (Chen and YiRage Contributors, 2026; AMD, 2024a).

1.9.1 Worked example: Llama-3.2-1B on Ryzen AI

Setup: batch-1, ctx=2048, hidden d=2048, GQA 16 Q / 4 KV heads (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Table 1.7: Llama-3.2-1B decode layer: operator-first vs static XDNA fusion (illustrative Ryzen AI cell).

Metric	Operator-first	Static fusion	Change
DMA edges/token (layer)	14	1	−92.9%
DDR activation bytes/token	∼49 KB	∼9 KB	−81.6%
Median ms/token (layer)	48 µs	22 µs	−54.2%
MAC active %	18%	22%	+22% (minor)
Watt/token (rel.)	1.00	0.62	−38%

Reading the worked row. Operator-first schedules pay DDR activation round trips *and* DMA orchestration tax (Chen, 2026; Vellaisamy *et al.*, 2026). Static fusion attacks both; MAC active % moves last—mirror Chapter 4's TC-utilization lesson on DDR (Davies *et al.*, 2025; NVIDIA, 2022). Triplet-compare the same IR on Hopper (Table **??** in Chapter 4) to keep fleet decisions honest (Chen and YiRage Contributors, 2026; Chen, 2020).

Common mistakes checklist. Importing unfused PyTorch ops one by one (Vellaisamy *et al.*, 2026; AMD, 2024b). Assuming dynamic batch/seq like CUDA (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Omitting softmax carrier fusion (Dao *et al.*, 2022; Chen, 2026). Skipping compile rejections because GPU numbers look fine (Davies *et al.*, 2025). Measuring prefill only (Dao *et al.*, 2023; AMD, 2024a). Each mistake reproduces Chapter 1 pathologies on different silicon (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Review question. A teammate proposes ping-pong DMA on every micro-copy but still stores attention output to DDR before MLP (Chen, 2026; Dao *et al.*, 2022). Ask: what happens to DDR bytes/token and carrier residency? (Chen and YiRage Contributors, 2026; AMD, 2024a) Correct answer: bytes stay high; static fusion legality fails despite pretty DMA timelines (Chen, 2026; Davies *et al.*, 2025).

Residual update placement. Final residual can be in-place in tile SRAM when dimensions align (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). A DDR residual write mid-layer is another $\Theta(d)$ export—flag in review (Chen, 2026, 2020).

GQA head accounting. Grouped-query attention reduces KV heads versus Q heads—KV loop traffic scales with KV head count (Kwon *et al.*, 2023; Dao

et al., 2022; Vellaisamy *et al.*, 2026). Worked example uses 16 Q / 4 KV; adjust tile budget line items for your architecture (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024a).

Unfused baseline narrative. Operator-first stacks pay TaxBreak launch counts on GPU *and* DDR activation bytes on XDNA (Vellaisamy *et al.*, 2026; Chen, 2026). Static fusion attacks both metrics on NPU; partial fusion attacks only one (Chen and YiRage Contributors, 2026; AMD, 2024b). Table 1.7 is the template for your model row—replace numbers, keep columns (Davies *et al.*, 2025; Chen, 2020).

Pattern usage in review. Require Pattern 1 skeleton diagram on every layer-fusion PR; Pattern 2 bucket map on every graph PR; Pattern 5 triplet diff on every IR change (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; AMD, 2024a).

1.9.2 Engineering patterns (XDNA decode)

Pattern 1: Static fused layer skeleton. Scene: single-layer decode on Ryzen AI (Vellaisamy *et al.*, 2026). **Solution:** DMA load → QKV MAC → KV loop with online softmax → SwiGLU → residual—no DDR activation stores (Dao *et al.*, 2022; AMD, 2024a). **Benefit:** ∼6–8 DDR round trips removed; DMA edges drop from ∼14 to 1 per layer (Chen, 2026). **Notes:** pre-compute tile SRAM budget (Table 1.6) (Chen and YiRage Contributors, 2026).

Pattern 2: Context buckets + static graphs. Scene: production ctx $\in \{512, 2048, 8192\}$ (Kwon *et al.*, 2023; AMD, 2024b). **Solution:** separate compiled graphs per bucket; padded max ctx inside each (Chen and YiRage Contributors, 2026). **Benefit:** deterministic decode latency; no runtime realloc (Davies *et al.*, 2025). **Notes:** bucket count trades storage for predictability (AMD, 2024a).

Pattern 3: Ping-pong DMA in KV loop. Scene: KV loop DDR-bound (SemiAnalysis, 2024; Dao *et al.*, 2023). **Solution:** paired buffer descriptors alternate K/V tiles while MAC consumes previous slot (AMD, 2024b; NVIDIA, 2022). **Benefit:** raises MAC active % without extra DDR traffic (Davies *et al.*, 2025). **Notes:** mis-timed DMA completion tokens stall the pipeline (Chen and YiRage Contributors, 2026).

Pattern 4: Spatial softmax carriers. **Scene:** attention decode (Dao *et al.*, 2022). **Solution:** fixed tile slots for (m, ℓ, o); spatial reduction tree (AMD, 2024a; Dao *et al.*, 2023). **Benefit:** avoids full attention map DDR materialization (Chen, 2026). **Notes:** Chapter 10 owns numerics; this chapter owns placement (Chen and YiRage Contributors, 2026).

Pattern 5: Triplet parity. **Scene:** fleet with GPU + NPU endpoints (Davies *et al.*, 2025; AMD, 2024b). **Solution:** same IR, three backends; compare normalized bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020). **Benefit:** prevents GPU-only schedule drift (Vellaisamy *et al.*, 2026; Chen, 2026).

Pattern 6: MoE static routing (advanced). **Scene:** MoE decode with expert selection (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). **Solution:** compile expert subgraphs + routing DMA table; batch experts where SRAM allows (AMD, 2024a; Chen and YiRage Contributors, 2026). **Benefit:** only path to legal MoE on static arrays (Davies *et al.*, 2025). **Notes:** hardest XDNA integration—profile at multiple ctx buckets (Kwon *et al.*, 2023).

1.10 XDNA Decode Benchmarking (Slim)

Chapter 1–2 define decode-cell metrics; this section adds *XDNA-specific* counters only (Vellaisamy *et al.*, 2026; Chen, 2026). Full harness checklists live in Appendix A (§**??**)—not repeated here (Chen and YiRage Contributors, 2026).

Four verification classes. **(1) Core metrics:** DDR bytes/token, DMA edges/token, ms/token and p99 at ctx $\in \{512, 2048, 8192\}$, BS=1 (Chen, 2026; Vellaisamy *et al.*, 2026). **(2) Environment:** pinned weights, Ryzen AI driver/runtime, shape buckets, warmup including graph init (AMD, 2024b,a). **(3) XDNA extensions:** MAC active %, tile SRAM high-water, compile time amortization (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). **(4) CI gate:** triplet legality + DDR bytes/token within tolerance or documented watt justification (Chen, 2020; AMD, 2024b).

Warm-up protocol. Include graph runner initialization and $\geq$1000 steady decode steps (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Report first-token and steady ms/token separately—laptop UX binds to both (Kwon *et al.*, 2023; AMD, 2024b).

Table 1.8: XDNA decode counter worksheet (extends Chapter 1).

Metric	XDNA decode meaning
DDR bytes/token	Weight + KV + spill traffic
DMA edges/token	Static orchestration tax
MAC active %	Tile utilization vs stall
Compile time	Amortization over session
p99 ms/token	Tail under shared DDR contention

Triplet runs. Export the same decoder IR to CUDA and XDNA; diff legality before celebrating either ms/token (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). A GPU launches/token win paired with XDNA DDR regression signals residency failure (Chen, 2026; Davies *et al.*, 2025).

Fleet parity. Laptop fleets share DDR with display, browser, and background services—isolated benches overstate steady-state goodput (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026). Qualify under realistic contention or publish contention assumptions in release notes (Chen and YiRage Contributors, 2026; AMD, 2024a).

Regression harness. Check in golden decode-cell scripts alongside static graphs: same tokenizer, ctx buckets, and DDR byte counters every CI run (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026; Vellaisamy *et al.*, 2026). Driver upgrades that change DMA timing without changing FLOPs still fail SLOs—treat runtime versions like CUDA toolkit pins (AMD, 2024a; Davies *et al.*, 2025).

Shape buckets. Compile separate static graphs for each $(\text{batch}, \text{ctx})$ class you ship; never extrapolate from a single prefill graph (AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Bucket count trades compile storage for decode predictability—document which bucket each SKU image selects at runtime (Chen and YiRage Contributors, 2026; AMD, 2024b).

Documentation contract. Every XDNA decode release note should list max context per bucket, compile time, steady ms/token, DDR bytes/token at ctx=2048, and watt/token when available (AMD, 2024b,a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026). Without published counters, partners cannot tell whether regressions come from graphs, drivers, or contention (Chen, 2020; Vellaisamy *et al.*, 2026).

Ablation discipline. Publish unfused, partially fused, and fully fused static graph rows—label which DDR metric moved (Chen, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a). Reviewers reject ms/token deltas that do not label which static rule was satisfied (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Paged KV stress. Benchmark with block tables when product ships paging—DMA legality at 32K may differ from 2K (Kwon *et al.*, 2023; Dao *et al.*, 2023; AMD, 2024b).

Handoff to Part IV. Chapter **??** generalizes residency budgeting across GPU, CPU, and NPU (Chen and YiRage Contributors, 2026; Chen, 2020). Carry XDNA counters defined here as mandatory fields in every cross-hardware decode report (AMD, 2024b,a; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Example 1.10.1. Ryzen AI decode gate: Llama-3.2-1B at ctx=2048 must show DDR bytes/token within tolerance *or* document watt/token justification; attach static buffer table and DMA edge list (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Compare with Chapter 4. Hopper Table **??** and XDNA Table 1.7 should originate from the same IR export (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026). Publish ms/token *and* watt/token when fleets mix silicon—hiding one metric misroutes product decisions (Davies *et al.*, 2025; AMD, 2024b,a; NVIDIA, 2022).

Regression hygiene. Pin tokenizer, weights, ctx bucket, driver, runtime, and YiRage commit for every comparison (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; AMD, 2024b). Record median and p99 ms/token; DDR bytes and DMA edges must move with latency (Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a).

What we deliberately omit here. Nightly driver pins, speculative-decode draft graphs, and multi-chip XDNA sweeps belong in appendix checklists and later fleet chapters—not in this hardware primer (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b). This section states verification *classes*; Appendix A hosts harness checkboxes (Chen and YiRage Contributors, 2026; Chen, 2020).

Closing rule. If DDR bytes/token and p99 ms/token both improve, the static graph change is real; if only FLOPs move, reject the merge (Chen, 2026; Vellaisamy *et al.*, 2026).

Practitioner mantra. Profile end-to-end decode first; cut DDR activation bytes second; collapse DMA edges third; tune MAC utilization fourth; re-profile on every driver and ctx bucket change (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; AMD, 2024b; Chen and YiRage Contributors, 2026). Teach teammates to read buffer tables before MAC assembly diffs (Chen, 2020; AMD, 2024a).

Security of performance claims. Never cite peak TOPS for decode chapters—cite named models, ctx buckets, and DDR bytes/token plus watt/token when available (Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b).

✓ **Practice checkpoint.** Promotion requires all three ctx buckets green on bytes/token *and* DMA edges/token within published tolerances (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a).

1.11 Key Takeaways

- **Static legality principle.** XDNA has no runtime scheduler—illegal buffer plans fail compile or DDR-bind execution (AMD, 2024b,a). **Action:** treat compile rejection as success; fix buffer tables before silicon (Chen and YiRage Contributors, 2026).
- **Metric principle.** Optimize DDR bytes/token and watt/token before MAC % polish (Chen, 2026; Davies *et al.*, 2025). **Action:** publish DDR ledger beside ms/token in every Ryzen AI release note (AMD, 2024b).
- **Translation principle.** Map CUDA concepts (TMA, graphs, carriers) but re-plan schedules for static tiles (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a). **Action:** use Table 1.3 in design review, not as a porting checklist (Chen and YiRage Contributors, 2026).
- **Fusion principle.** Online softmax carriers must stay in tile SRAM for the full attention pipeline (Dao *et al.*, 2022, 2023). **Action:** block merges that add a DDR softmax materialization edge (Chen, 2026).

- **Verification principle.** Triplet IR must compile on CUDA and XDNA; bytes/token must move together with latency (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026). **Action:** attach Table 1.8 rows and five review gates to CI artifacts (Davies *et al.*, 2025).

1.12 Conclusion

AMD XDNA/AIE is the static counterpart to Chapter 4's dynamic Hopper: fixed tile SRAM, compile-time DMA graphs, and deterministic decode at the cost of flexibility (AMD, 2024b,a; NVIDIA, 2022). The unified lesson across both chapters is mechanical sympathy—cut activation egress and orchestration before tuning compute utilization (Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020).

Chapter **??** generalizes on-chip residency budgeting across GPU, CPU, and NPU (Chen and YiRage Contributors, 2026). Carry forward DDR bytes/token, DMA edges/token, and tile SRAM worksheets from this chapter into every cross-hardware decode report (Davies *et al.*, 2025; AMD, 2024b).

Worksheet gate. Before Part IV MegaKernel chapters, confirm Table 1.2 is satisfied for your model and all five review gates pass (Chen and YiRage Contributors, 2026; AMD, 2024a). If DDR bytes/token is flat on XDNA, Chapter 6 will not rescue a GPU-transplanted schedule (Chen, 2020, 2026).

Hands-on next step. Pick one decode layer, fill Table 1.7 for your model, and mark which static rule still has headroom (Davies *et al.*, 2025; AMD, 2024b). If DDR bytes/token is flat, return to Chapter 2 export edges before studying new Ryzen AI SKUs (Chen, 2020, 2026; Chen and YiRage Contributors, 2026).

Summary before leaving Part II. Tiles, static rules, CUDA mapping, softmax placement, tradeoffs, YiRage gates, patterns, worked numbers, and slim verification form one mechanical-sympathy story for spatial dataflow (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). DDR bytes first, DMA edges second, MAC last—when the DDR roofline binds at batch-1 decode (Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024b). Chapter 4 and this chapter together define the GPU/NPU bookends for Part IV residency work (Chen and YiRage Contributors, 2026; Chen, 2020).

Glossary carry-forward. **XDNA**, **AIE**, **buffer descriptor**, and **static dataflow** are defined on first use in this chapter; later NPU and compiler chapters reuse them without re-deriving (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2020).

GPU–NPU symmetry recap. Hopper rewards dynamic residency within SM limits; XDNA rewards static residency with zero DDR exports (NVIDIA, 2022; AMD, 2024b; SemiAnalysis, 2024). Triplet bufferization exists because both truths must hold from one IR (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Skillful engineering. Mid-tier skillful static fusion beats flagship silicon running DDR-bound eager loops—the same lesson as Memory-Floor L4 versus H100 cells, transplanted to watt-capped edge (Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Hands-on recap. (1) Fill Table 1.2 for your model. (2) Build Table 1.6. (3) Run Table 1.7 counters. (4) Pass five review gates. (5) Export triplet legality before merge (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b). If any step is missing, the PR is not XDNA-ready (Chen, 2020; AMD, 2024a).

Part II closing. Chapters 4–5 complete the GPU/NPU hardware depth needed before Part IV residency passes (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; NVIDIA, 2022). Read Chapter 6 next with the same worksheet—CPU cache blocking punishes the same export mistakes with a third legality mechanic (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Done criteria. You are done when you can explain why a Hopper fusion illegalizes on XDNA and which DDR counter proves your static graph is shippable (Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a). Proceed to Chapter **??** with completed tile SRAM and DDR worksheets (Chen, 2020; Chen and YiRage Contributors, 2026).

Metric reminder. DDR bytes/token, DMA edges/token, and ms/token must move together for a shippable XDNA change (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). MAC active % alone is never sufficient evidence (AMD, 2024a; Chen and YiRage Contributors, 2026).

Closing artifact. Archive shape bucket, buffer table, Table 1.7 row, and DDR screenshot set beside each merged XDNA PR (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b). Future you—and Chapter 6 reviewers—need that packet to judge triplet parity (Chen, 2020; AMD, 2024a).

One-sentence chapter summary. XDNA decode optimization is static legality first—tile SRAM and DDR bytes/token before MAC polish—with the same export-edge mindset as GPU, but zero runtime forgiveness (AMD, 2024b,a; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Version note. Future Ryzen AI steppings extend tile counts and SRAM rows; this chapter's static rules and review gates remain valid when chip-model tables update (AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a; Chen, 2020).

Index terms for later lookup. Readers searching the index should start from XDNA, AIE, dataflow, and static graph entries—they point back to this chapter's rules, mapping table, and worksheets (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026).

Final synthesis. AMD XDNA gives spatial MAC arrays and deterministic DMA—tools for mechanical sympathy, not automatic speedups (AMD, 2024b,a; Davies *et al.*, 2025; Liu *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026). Measure DDR goodput; co-design static residency; gate merges with buffer tables and five review checks (Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b,a).

Chapter artifact. Maintain an XDNA worksheet: shape bucket targeted, tile SRAM line items, Table 1.7 row, and DDR counter cheat sheet fields (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; AMD, 2024b,a). Update on every ctx bucket or driver change (Kwon *et al.*, 2023; Chen, 2020).

Bibliography

AMD (2024a). Ai engine technology. https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29

AMD (2024b). AMD XDNA architecture. https://www.AMD.com/en/technologies/XDNA.html. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 24, 25, 26, 27, 28, 29

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 6, 7, 8, 9, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. https://github.com/chenxingqiang/YiRage. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 3, 6, 7, 9, 12, 13, 16, 17, 19, 20, 21, 22, 23, 25, 26

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 16, 17, 19, 20, 21, 22, 23, 26, 27, 28, 29

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 3, 4, 5, 6, 17, 26, 27, 28, 29

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 18

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 3, 4, 7, 8, 9, 11, 14, 16, 17, 20, 21, 22, 23, 24, 25, 29

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 3, 4, 11, 14, 16, 23, 25, 29

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 4, 5, 10, 11, 16, 21, 22, 25, 26, 27, 28, 29

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 2, 6, 9, 11, 12, 13, 14, 19

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 4, 10, 11, 12, 13, 14, 16, 17, 22, 26, 27, 28, 29

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29

Index

AIE, 4

dataflow, 4

XDNA, 4